

PL/ SQL EXERCISES

Assumptions

Since the database schema was not provided in the assignment, the following tables were created to implement and test the PL/SQL solutions.

Accounts Table

```
CREATE TABLE Accounts (  
    AccountID NUMBER,  
    CustomerName VARCHAR2(50),  
    Balance NUMBER,  
    AccountType VARCHAR2(20)  
);
```

Employees Table

```
CREATE TABLE Employees (  
    EmployeeID NUMBER,  
    EmployeeName VARCHAR2(50),  
    Department VARCHAR2(30),  
    Salary NUMBER  
);
```

Sample Data:

Query result Script output DBMS output Explain Plan SQL history				
  Download ▾ Execution time: 0.156 seconds				
	ACCOUNTID	CUSTOMERNAME	BALANCE	ACCOUNTTYPE
1	101	John	10000	Savings
2	102	Alice	15000	Savings
3	103	Bob	8000	Current
4	104	Emma	20000	Savings

Query result Script output DBMS output Explain Plan SQL history				
  Download ▾ Execution time: 0.011 seconds				
	EMPLOYEEID	EMPLOYEENAME	DEPARTMENT	SALARY
1	1	Rahul	IT	50000
2	2	Priya	IT	60000
3	3	Amit	HR	45000
4	4	Sneha	Finance	55000

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question: Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.**

PROCEDURE:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
END;
/
```

EXECUTE PROCEDURE:

```
BEGIN
    ProcessMonthlyInterest;
END;
/
```

VERIFICATION:

```
SELECT * FROM Accounts;
```

OUTPUT:

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.001 seconds					
	ACCOUNTID	CUSTOMERNAME	BALANCE	ACCOUNTTYPE	
1	101	John	10100	Savings	
2	102	Alice	15150	Savings	
3	103	Bob	8000	Current	Savings
4	104	Emma	20200	Savings	

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question: Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.**

PROCEDURE:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus
(
  p_department IN VARCHAR2,
  p_bonus_percent IN NUMBER
)
IS
BEGIN
  UPDATE Employees
  SET Salary = Salary + (Salary * p_bonus_percent / 100)
  WHERE Department = p_department;

  COMMIT;
END;
/
```

EXECUTE PROCEDURE:

```
BEGIN
  UpdateEmployeeBonus('IT', 10);
END;
/
```

VERIFICATION:

```
SELECT * FROM Employees;
```

OUTPUT:

Query result Script output DBMS output Explain Plan SQL history				
  Download ▾ Execution time: 0.004 seconds				
	EMPLOYEEID	EMPLOYEEENAME	DEPARTMENT	SALARY
1	1	Rahul	IT	55000
2	2	Priya	IT	66000
3	3	Amit	HR	45000
4	4	Sneha	Finance	55000

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.**

PROCEDURE:

```
CREATE OR REPLACE PROCEDURE TransferFunds
(
  p_from_account IN NUMBER,
  p_to_account IN NUMBER,
  p_amount IN NUMBER
)
IS
  v_balance NUMBER;
BEGIN

  SELECT Balance
  INTO v_balance
  FROM Accounts
  WHERE AccountID = p_from_account;
```

```

IF v_balance >= p_amount THEN

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Transfer Successful');

ELSE

    DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

END IF;

END;
/

EXECUTE PROCEDURE:
BEGIN
    TransferFunds(101, 102, 2000);
END;
/

VERIFICATION:
SELECT * FROM Accounts;

```

OUTPUT:

Query resultScript outputDBMS outputExplain PlanSQL history

Download

Execution time: 0.004 seconds

	ACCOUNTID	CUSTOMERNAME	BALANCE	ACCOUNTTYPE
1	101	John	8100	Savings
2	102	Alice	17150	Savings
3	103	Bob	8000	Current
4	104	Emma	20200	Savings

Savings